

• Section-A (2x5=10)

(1) Find the tokens for the given code:

```
printf("i = %d, &i = %x", i, &i);
```

Ans Tokens in this statement are:

- printf - (keyword / Identifier)
- (- Left Parenthesis
- "i = %d, &i = %x" - String Literal
- , - Comma
- & - Address Operator
- i - Identifier
-) - Right Parenthesis
- ; - Semicolon

2. What is input buffering?

Ans Input buffering is a technique used in lexical analysis to optimize the scanning of input characters. It helps reduce the number of disk I/O operations by storing a block of characters in memory before processing them.

3. What is backpatching?

Ans Backpatching is a method used in compiler design for handling forward ~~leaps~~ jumps in intermediate code generation. It is used in syntax-directed translation to delay the address resolution of jump statements until the target labels are determined. This is commonly used in code generation for conditional and loop statements.

4. What is bootstrapping?

Ans Bootstrapping in compiler design refers to the process of using a simple compiler to build a more advanced compiler. It involves writing a compiler in the same language it compiles (self-hosting). This technique is useful for porting compilers to new platforms.

5. Difference between single-pass and multi-pass compilers.

- Ans:
- Single-pass compiler - Processes the source code in one pass without revisiting earlier parts. It is faster but may produce less optimized code. (e.g., early assembly language compilers).
 - Multi-pass compiler - Reads the source code multiple times, allowing for better optimization and error checking (e.g., modern C & Java compilers). It divides compilation into multiple phases like lexical analysis, syntax analysis, ~~and~~ semantic analysis.

• Section-B ($3 \times 7 = 21$)

1. Why do we need a syntax tree while constructing a compiler.

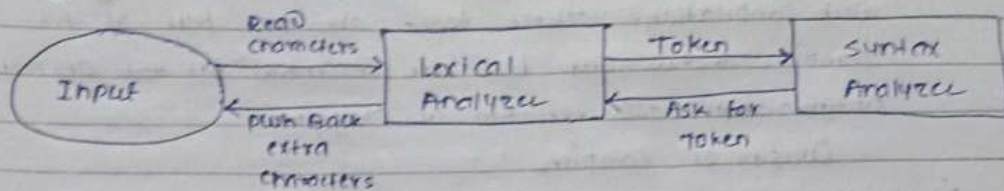
Ans: A syntax tree (abstract syntax tree or AST) is essential in compiler construction because -

- It represents the hierarchical structure of the source code.
- Simplifies semantic analysis by eliminating unnecessary grammar details.
- Helps in optimization by structuring operations efficiently.
- Helps in code generation, making translation to machine code easier.
- Supports error handling by providing a structured format for debugging.
- Enables transformations like constant folding and loop unrolling.
- Used in interpreters and compilers for efficient program execution.

2. Explain lexical analyzer generator (LEX) and its structure.

Ans: A lexical analyzer generator (LEX) is a tool used to create a lexical analyzer, which is a crucial component of a compiler. The lexical analyzer is responsible for breaking down the input source code into meaningful

tokens, which are then passed to the parser for further processing. LEX automates the creation of this Lexical by generating code (often in C) based on a set of rules defined by the user.



• Structure of LEX :-

(i) Definition Section - This section includes declarations of variables, constants, and regular expressions that will be used in the rules section.

```

ex: %s include <stdio.h> %}
    %c [0-9]
    %c LETTER (a-zA-Z)
  
```

(ii) Rules Section - This section contains the patterns (regular expressions) and the corresponding action (C code) that the Lex will execute when a pattern is matched.

```

ex: - {DIGIT}+ { printf ("Integer: %s\n", yytext); }
    {LETTER}+ { printf ("Identifier: %s\n", yytext); }
  
```

(iii) Auxiliary Functions - (Users Subroutine Section): - This section contains additional C code that can be used by the Lex such as helper functions. It is optional and can be omitted if not needed.

```

ex: - int main() {
        yylval();
        return 0;
    }
  
```


Q. What do you understand by the terms cousins of a compiler?

Ans The term "cousins of a compiler" refers to a set of related programs or tools assist in the various stages of the compilation process or in tasks closely associated with compilation. These tools are not part of the core compiler of the ~~for~~ but are essential for the development, optimization and execution of programs.

Cousins of compiler :-

- (i) Preprocessor - Handles macros, header files, and directives before compilation.
- (ii) Assembler - Converts assembly code into machine code.
- (iii) Linker - Combines object files into a single executable.
- (iv) Loader - Loads executable code into memory for execution.
- (v) Interpreter - Directly executes source code without compilation.
- (vi) Debugger - Helps find and fix errors in the program.
- (vii) Profiler - Analyzes program performance and resource usage.
- (viii) Editor - A text editor is used to write and modify the source code.
- (ix) Version Control system (VCS) - VCS helps manage changes to the source code over time.

• Section - C (3x11=33)

Q. Discuss compilation process. Write down the output of each phase for the expression $a = b * c + d / e$

Ans The compilation process is the sequence of steps a compiler follows to translate high-level source code into machine code that can be executed by a computer. Each phase of the compilation process transforms the input in some way, ultimately producing

An executable program.

The compilation process involves several phases:

(i) Lexical Analysis -

- Converts source code into tokens.

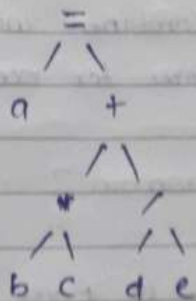
ex:

Identifier (a),	Operator (+),
Assignment (=),	Identifier (d),
Identifier (b),	Operator (/),
Operator (*),	Identifier (e),
Identifier (c),	

(ii) Syntax Analysis (Parsing) -

- Constructs a parse tree based on grammar.

ex:



(iii) Semantic Analysis -

- Checks for semantic errors like type mismatches.
- Annotated AST with semantic information (eg data types).

(iv) Intermediate Code Generation -

- Converts into three-address code (TAC).
- Like postfix notation.

ex:

$t_1 = b * c$

$t_2 = d / e$

$t_3 = t_1 + t_2$

$a = t_3$

(v) Optimization -

- Remove unnecessary computations for efficiency.

- (vi) Code Generation - Translates the optimized intermediate code into target machine code.

ex:

```

LOAD R1, b
LOAD R2, c
MUL R3, R1, R2 ; R3 = b * c
LOAD R4, d
LOAD R5, e
DIV R6, R4, R5 ; R6 = d/e
ADD R7, R3, R6 ; R7 = R3 + R6
STORE a, R7 ; a = R7

```

- (vii) Linking and Loading - Combined with libraries, load into memory and prepare for execution.

2. (a) What is an ambiguous Grammar? Specify the demerits of ambiguous Grammar.

- (b) What are the various types of errors in compilation process? Explain each type with examples.

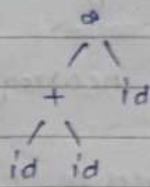
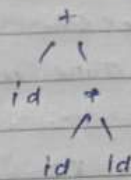
Ans: (a) An ambiguous Grammar is a context free Grammar (CFG) in which a string (or sentence) can have more than one parse tree or more than one leftmost/rightmost derivation. This means that the same input can be interpreted in multiple ways leading to different syntactic structures.

ex.

$$E \rightarrow E + E \mid E * E \mid id$$

Here E is the start symbol and id represent an identifier.

For the input $id + id * id$ this grammar allows two different parse trees.

Interpretation 1: $(id + id) * id$ Interpretation 2: $id + (id * id)$ 

• Demerits :-

- Causes multiple interpretations of the same statement.
- Leads to incorrect parsing and compilation errors.
- Affects optimization and semantic analysis.

(b) During the compilation process, various types of error can occur at different stages. These errors prevent the successful compilation and execution of a program. The error can be categorized into many types -

(i) Lexical Errors - Error that occurs during the Lexical analysis phase when the compiler cannot recognize a sequence of characters as a valid token.

- Causes :- invalid character, misspelled keyword or identifiers.

ex: `int a@b = 10; // '@' is not valid character`
`int x = 5; // itn is a misspelling.`

(ii) Syntax Errors - Errors that occur during the syntax analysis phase when the source code violates the grammar rules of the programming language.

- Causes - missing or misplaced punctuation (semicolons, braces, ...)
- incorrect use of operators or keywords.

ex: `int x = 10; // missing semicolon`
`printf("%d", x);`

`int x = 10;`

`x = x + 1; // incomplete expression.`

(iii) Semantic error :- Occurs when meaning is incorrect.

• Causes :-

Type mismatches, Undeclared or Incorrectly used variables.

eg `int x = "Hello";` // Assigning a string to an integer variable.

(iv) Runtime Errors :- Occurs during execution.

Example: `int x = 5/0;` // Division by zero.

(v) Logical Error :- Occurs due to incorrect logic.

eg `if (x > y) x = y;` // wrong condition.

3. What is Error Handling? Discuss in detail.

Ans Error handling refers to the methods used to detect, report and recover from errors during compilation or execution.

• Types of Error Handling in Compilation -

(i) Panic mode Recovery - Skips erroneous statement and continues compilation.

eg ignoring "missing semicolon".

(ii) Phrase - Level Recovery - Tries to correct the error and continue.
eg Adding missing '}' automatically.

(iii) Error Productions: - Modifies Grammar to catch errors.

eg - Adding a rule to detect if (expr without)

(iv) Global Correction: - Tries to find the least changes needed to compile.
eg Suggesting 'int' instead of float for `int x = 5.0;`

- Phase of Error Handling - / Technician for Error Handling - error handling involves the following phases:-

- (i) Error Detection - The compiler identifies errors in the source code during lexical analysis, syntax analysis, semantic analysis and linking.
- (ii) Error Reporting - The compiler generates meaningful error messages that describe the type and location of the error.
- (iii) Error Recovery - The compiler attempts to recover from the error and continue processing the code to detect additional errors.

ex:- #include <stdio.h>

int main() {

int x = 10

int y = "Hello";

printf("%d", x+y);

return 0;

}

- Lexical Error - No lexical errors.

- Syntax Analysis - Error: missing semicolon after "int x = 10"

Recovery - compiler inserts a semicolon.

- Semantic Analysis - Error: Type mismatch (int y = "Hello").

Recovery - compiler reports error and skips the statement.

- Code Generation - The compiler generates code for the valid statements.

- Linking - No linker error.